Credit Card Fraud Detection

Course: CPT_S 315

Team Members: Jack Underhill, Beck Williams, Clayton Simoneaux, John Cha

Repository: https://github.com/Data-Mining-G5/Fraud-Detection

Contribution Statement

The completed project work was divided equally among the three active contributors. Beck Williams is listed as a team member but is assigned 0% because he has not contributed to the project or communicated with the team since the abstract assignment.

Team Member	Contribution	Main Contribution
Jack Underhill	33.34%	Model comparison, threshold tuning, final evaluation, notebook/report integration, and project coordination
Clayton Simoneaux	33.33%	Exploratory feature analysis, fraud/non-fraud comparison, correlation and indicator-rule analysis, and Role 2 output artifacts
John Cha	33.33%	Data preprocessing, duplicate/null/class-balance checks, stratified train/validation/test splitting, and scaled processed data artifacts
Beck Williams	0.00%	No project contribution or team communication after the abstract assignment

Abstract

This project studies credit card fraud detection as a highly imbalanced binary classification task in which missed fraud and false alarms have different real-world costs. Using the Kaggle Credit Card Fraud Detection dataset, we cleaned duplicate transactions, created stratified train, validation, and test splits, scaled non-anonymized numerical features, explored fraud-related feature patterns, compared Logistic Regression, Random Forest, Gradient Boosting, class-weighted, and undersampled models, and tuned the final decision threshold on validation data. Random Forest produced the strongest validation performance and, after selecting a threshold of 0.40, achieved 0.9324 precision, 0.7263 recall, 0.8166 F1 score, and 0.7627 PR-AUC on the held-out test set, detecting 69 of 95 fraud cases with only 5 false positives. These results show that fraud detection should be judged with precision-recall metrics and confusion matrix trade-offs rather than accuracy alone, especially when fraudulent transactions represent less than 0.2% of the data.

1. Introduction

Credit card fraud detection is a practical data mining problem in digital payment systems. A missed fraudulent transaction can result in direct financial loss, while an incorrect fraud alert can block legitimate customer activity, frustrate customers, and increase the manual review workload. This makes the task more complicated than ordinary classification because the most useful model is not simply the model with the highest overall accuracy. Instead, the project must consider the trade-off between finding fraud and limiting false alarms.

This project uses the Kaggle Credit Card Fraud Detection dataset, which contains European credit card transactions from September 2013. Fraud is rare in the dataset, representing about 0.17% of transactions. This rarity poses the main technical challenge: a model can appear accurate by predicting nearly every transaction as non-fraud, yet still fail at the actual goal of identifying fraudulent cases.

As a team, we chose this project because fraud detection is a real-world example of data mining where the consequences of different errors are not equal. In many classification problems, a mistake is treated as simply wrong, but in fraud detection, a false negative can allow fraud to pass through, while a false positive can interrupt a legitimate customer and create extra review work. This made the project a useful way to connect classroom classification methods to an applied problem with practical costs. Our goals were to build a reproducible workflow, compare several reasonable model choices, study how severe class imbalance changes evaluation, and choose a decision threshold that reflects the trade-off between missed fraud and false alarms.

The main questions investigated in this project were:

1. Which transaction features or transformed components differ most between fraud and non-fraud cases?
2. Which model family performs best for detecting rare fraud cases?
3. Do class weighting or undersampling improve performance under severe class imbalance?
4. How does changing the decision threshold affect precision, recall, F1 score, and confusion matrix results?
5. What final trade-off between missed fraud and false positives is most defensible for this dataset?

Briefly, the project found that Random Forest performed best among the tested models on validation PR-AUC. After threshold tuning, the selected threshold was 0.40. On the held-out test set, the final model reached precision 0.9324, recall 0.7263, F1 0.8166, and PR-AUC 0.7627, with 69 detected fraud cases, 26 missed fraud cases, and 5 false positives.

2. Data Mining Task

This project treats fraud detection as a supervised binary classification task. Each input example is one credit card transaction represented by “Time”, “Amount”, and anonymized PCA-transformed variables V1 through V28. The output is a predicted label, where “Class = 0”

represents a legitimate transaction and “Class = 1” represents fraud. The models also produce fraud probabilities, which are used to choose a decision threshold.

The task can be stated as follows: given a transaction's numerical feature vector, estimate whether the transaction is fraudulent. For example, if the final model assigns a transaction a fraud probability greater than or equal to the selected threshold of 0.40, the transaction is classified as fraud. If the probability is below that threshold, it is classified as non-fraud.

The main challenge is that fraud is extremely rare. After cleaning, only 473 of 283,726 transactions are labeled fraud. This means accuracy is not enough to judge the model. A classifier that predicts every transaction as non-fraud would be more than 99% accurate but useless for fraud detection. The project, therefore, focuses on precision, recall, F1 score, PR-AUC, and confusion matrix counts. Another challenge is interpretability: most features are PCA-transformed and anonymized, so the project can identify predictive components but cannot always explain them in ordinary transaction terms.

3. Technical Approach

The project workflow is organized into four role-based scripts and one main notebook. The notebook runs the scripts in order, while the scripts contain the reusable implementation. The overall pipeline is:

1. Load the raw Kaggle dataset.
2. Check for duplicate rows, null values, and class imbalance.
3. Remove duplicate transactions.
4. Create stratified train, validation, and test splits.
5. Scale “Time” and “Amount”.
6. Perform exploratory feature analysis.
7. Compare models on the validation set.
8. Tune the decision threshold on the validation set.
9. Evaluate the final model on the held-out test set.

The diagram below (Fig. 1) shows the general workflow of the approach.

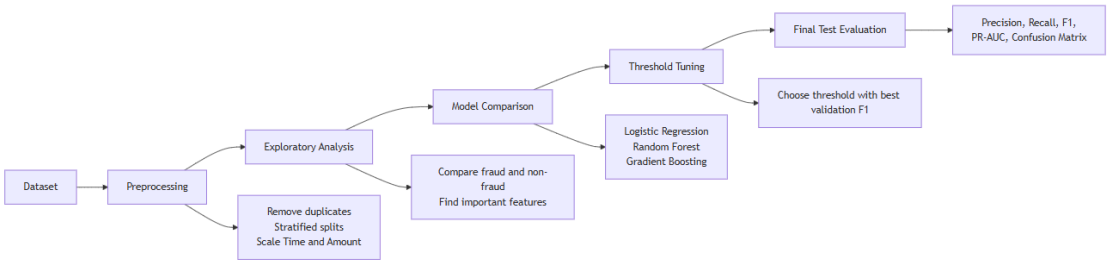

Figure 1. Technical approach workflow diagram

The preprocessing step loads the Kaggle archive, verifies that required fields are present, checks for null values, removes duplicates, creates stratified train/validation/test splits, and scales “Time” and “Amount” with a “StandardScaler” fit only on the training split. Fitting the scaler only on training data avoids leaking validation or test information into training. The processed splits are then saved for the later roles.

Item	Value
Raw rows	284,807
Duplicate rows removed	1,081
Clean rows	283,726
Null values	0
Fraud rows after cleaning	473
Non-fraud rows after cleaning	283,253
Fraud rate after cleaning	0.1667%

Split	Rows	Non-fraud	Fraud	Fraud rate
Train	170,234	169,951	283	0.1662%
Validation	56,746	56,651	95	0.1674%
Test	56,746	56,651	95	0.1674%

Exploratory analysis compared fraud and non-fraud records, measured feature correlations with the class label, identified high-lift indicator rules, and used an exploratory balanced logistic regression model for feature importance and validation error review. The strongest fraud-related

components included “V17”, “V14”, “V12”, “V10”, “V16”, “V3”, and “V7”. Since these variables are anonymized PCA components, the analysis is useful for modeling but limited for direct business interpretation.

The modeling step compared logistic regression, class-weighted logistic regression, random forest, class-weighted random forest, gradient boosting, and a logistic regression model trained on a 10:1 undersampled training set. These models were first evaluated on the validation split at the default threshold of 0.5. The test split was not used during this selection stage.

The final evaluation step selected Random Forest based on validation PR-AUC, then tuned only the decision threshold on the validation set. Thresholds from 0.05 through 0.95 were tested in increments of 0.05, and the selected threshold was the one with the best validation F1 score. This produced a selected threshold of 0.40. After selecting the threshold, the model was evaluated once on the held-out test split.

4. Evaluation Methodology

The dataset comes from the Kaggle Credit Card Fraud Detection dataset [1], based on European cardholder transactions from September 2013. The original dataset contains 284,807 transactions and 492 fraud cases before duplicate removal. Features “V1” through “V28” are PCA-transformed for confidentiality, while “Time”, “Amount”, and “Class” remain directly described.

The evaluation design separates training, validation, and test responsibilities. The training split is used to fit models. The validation split is used for model comparison and decision threshold tuning. The test split is used once after the model and threshold have already been selected. This reduces the risk of overestimating performance by repeatedly adapting decisions to the final test data.

The main metrics are precision, recall, F1 score, PR-AUC, and confusion matrix counts. Precision measures how many flagged transactions are actually fraud, which matters because false positives create customer friction and manual review costs. Recall measures how many real fraud cases are caught, which matters because false negatives represent missed fraud. F1 score balances precision and recall. PR-AUC measures precision-recall behavior across thresholds and is more appropriate than accuracy for rare-event detection. Accuracy is still reported for completeness, but it is not treated as the primary success metric.

5. Results and Discussion

The preprocessing results confirmed that the dataset had no null values, but duplicate transactions were removed before splitting. The cleaned data remained extremely imbalanced, with fraud making up only 0.1667% of transactions. This class balance shaped the rest of the project because it made accuracy a poor measure of success.

Exploratory analysis showed that fraud cases differed from non-fraud cases on several anonymized PCA components. The strongest correlations with fraud included “V17”, “V14”, “V12”, “V10”, “V16”, “V3”, and “V7” as shown below (Fig. 2).

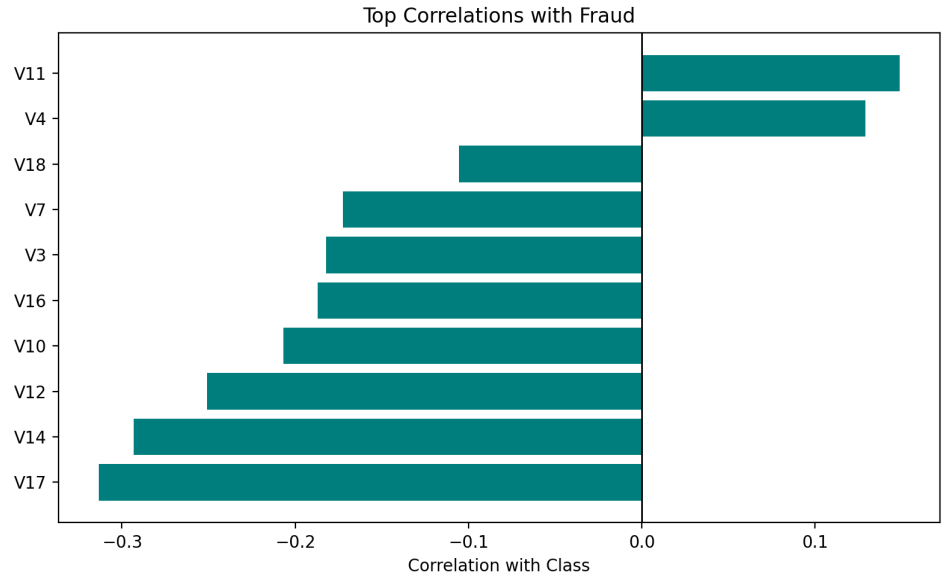

Fig. 2: Top correlation features with fraud class

Indicator rules based on extreme feature values also showed much higher fraud rates than the baseline rate. These findings suggested that fraud cases were not random noise in the feature space; they occupied detectable regions, even though the anonymized features limited direct interpretation.

The validation model comparison is shown below (Fig. 3). Random Forest had the strongest validation PR-AUC and F1 at the default threshold, so it was selected for final threshold tuning.

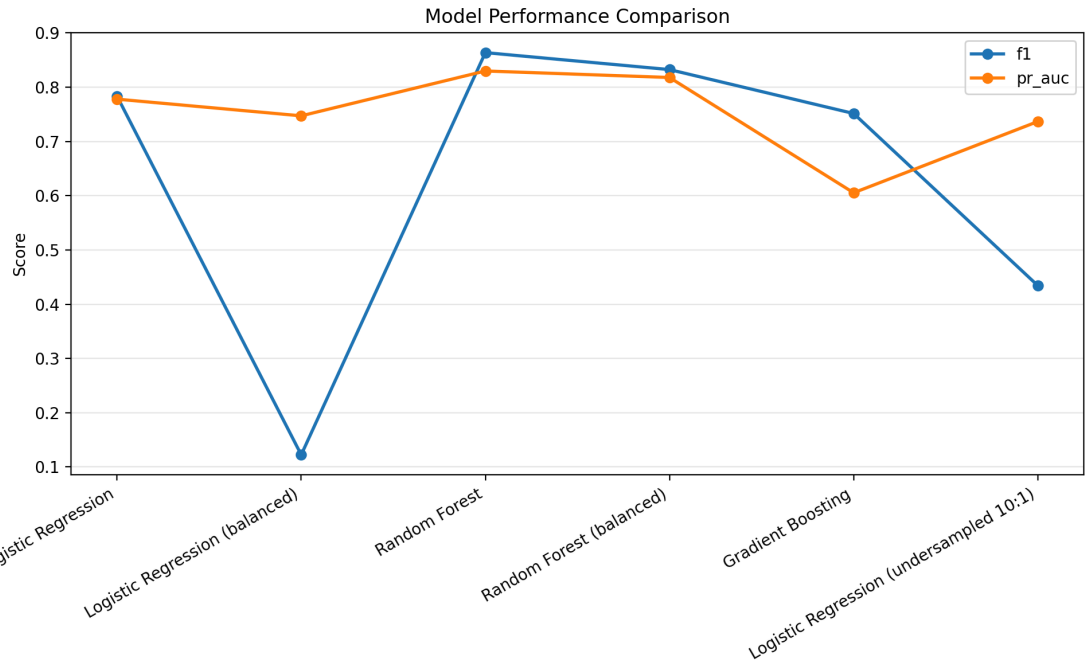

Figure 3. The figure shows the performance of the different models

Model	Precision	Recall	F1	PR-AUC	FP	FN	TP
Logistic Regression	0.9155	0.6842	0.7831	0.7779	6	30	65
Logistic Regression (balanced)	0.0657	0.8947	0.1224	0.7471	1,209	10	85
Random Forest	0.9383	0.8000	0.8636	0.8298	5	19	76
Random Forest (balanced)	0.9231	0.7579	0.8324	0.8178	6	23	72
Gradient Boosting	0.8333	0.6842	0.7514	0.6053	13	30	65
Logistic Regression (undersampled 10:1)	0.2914	0.8526	0.4343	0.7366	197	14	81

The comparison also shows why imbalance handling needs to be judged carefully. Balanced logistic regression and undersampling increased recall, but they produced many more false positives. In fraud detection, this may be unacceptable because every false positive can represent a blocked legitimate transaction or an additional manual review. Random Forest gave the strongest balance between precision and recall among the tested models.

Threshold tuning further refined this trade-off. The default threshold of 0.50 was not automatically assumed to be best. Instead, the Random Forest fraud probabilities were converted to predictions using a validation threshold grid. The most important thresholds are shown below (Fig. 4).

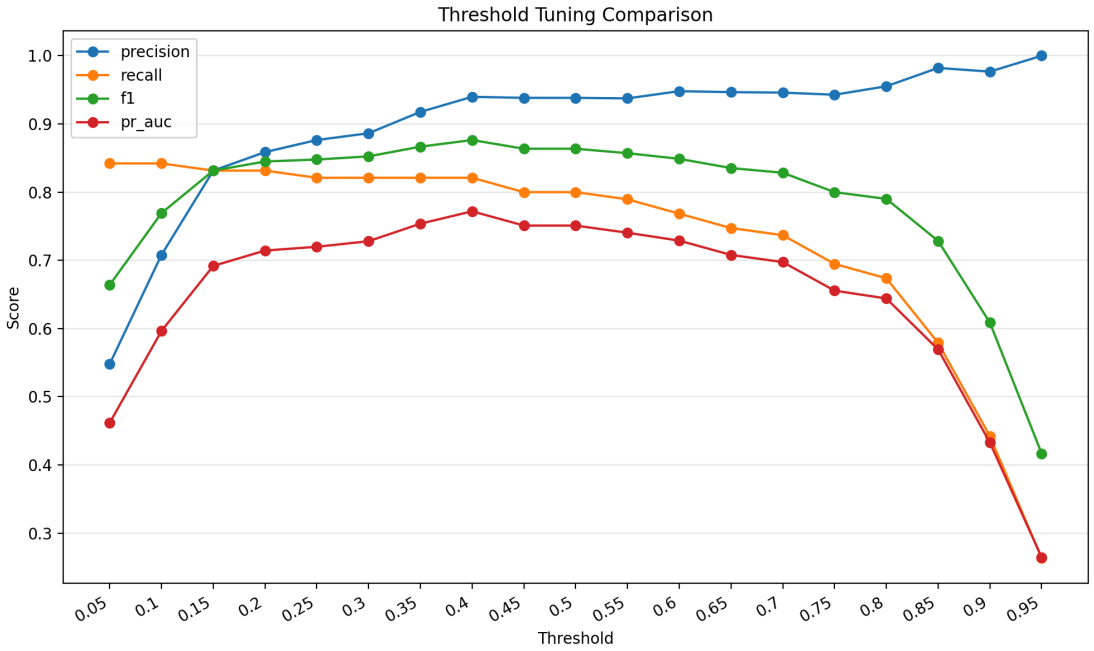

Fig. 4: Feature comparison between thresholds

Threshold	Precision	Recall	F1	PR-AUC	TN	FP	FN	TP
0.35	0.917647	0.821053	0.866667	0.753736	56644	7	17	78
0.40	0.939759	0.821053	0.876404	0.771891	56646	5	17	78
0.45	0.938272	0.800000	0.863636	0.750952	56646	5	19	76
0.50	0.938272	0.800000	0.863636	0.750952	56646	5	19	76

Threshold 0.40 produced the best validation F1 score. Compared with the threshold of 0.50, it caught two additional validation fraud cases without increasing false positives. Lower thresholds caught more fraud in some cases but created more false positives, while higher thresholds missed more fraud.

The final held-out test results are shown below. This test evaluation was performed after choosing the model and threshold from validation data.

Threshold	Precision	Recall	F1	PR-AUC	Accuracy	TN	FP	FN	TP
0.40	0.9324	0.7263	0.8166	0.7627	0.9995	56,646	5	26	69

The final model produced very few false positives: only 5 legitimate transactions were flagged as fraud out of 56,651 non-fraud test transactions. However, recall was lower on the test split than on validation, with 26 of 95 fraud cases missed. This result shows the central trade-off in fraud detection. The model was conservative and precise, but it did not catch every fraud case. In a real payment system, whether this threshold is acceptable would depend on the relative cost of missed fraud, customer friction, and manual review.

Overall, the parts that worked best were the stratified splitting strategy, the use of PR-AUC and F1 rather than accuracy, and threshold tuning after model selection. The main limitations were the extreme class imbalance, the reduced interpretability caused by anonymized PCA features, and the fact that the model still missed some fraud cases. A real fraud detection system would also need ongoing monitoring because fraud patterns can change over time.

6. Algorithm Selection and Relation to Prior Work

Although this project was an applied data mining project rather than a direct reproduction of one paper, our algorithm choices and evaluation design were still guided by common approaches used in prior work on imbalanced fraud detection. The main connection to prior work is the focus on rare event classification, imbalance handling, probability thresholds, and precision-recall-based evaluation.

We compared three main supervised classification families: Logistic Regression, Random Forest, and Gradient Boosting. Logistic Regression was chosen as a clear baseline because it is simple, fast, and commonly used for binary classification. Random Forest was chosen because it can model nonlinear relationships and feature interactions in tabular data without requiring heavy preprocessing beyond numerical scaling. Gradient Boosting was included as another strong tabular-data method so that Random Forest could be compared against a different tree-based ensemble approach.

Because fraud represented less than 0.2% of the cleaned dataset, we also tested imbalance-aware variants. These included class-weighted Logistic Regression, class-weighted Random Forest, and Logistic Regression trained on a 10:1 undersampled training set. These

experiments helped answer one of the project's central questions: whether imbalance handling improves fraud detection enough to justify any increase in false positives. The results showed that class weighting and undersampling often increased recall, but they also created many more false positives, especially for balanced Logistic Regression and the undersampled Logistic Regression model.

This result is consistent with a broader lesson from prior work on credit card fraud detection: overall accuracy is not the best way to judge rare-event classification, and the decision threshold can matter as much as the model family. Dal Pozzolo et al. [2], for example, discuss probability calibration and undersampling for unbalanced classification. While our project did not reproduce their exact experimental setup, it investigated a related practical issue: how model choice, imbalance handling, and threshold selection affect precision, recall, F1 score, PR-AUC, and confusion matrix outcomes.

The main observation from our comparison was that the standard Random Forest model provided the best validation PR-AUC and the strongest balance between precision and recall among the tested models. After threshold tuning, it remained conservative on the held-out test set: it produced only 5 false positives but missed 26 of 95 fraud cases. This reinforces the central finding of our project and related fraud-detection work: the best model is not simply the one with the highest accuracy, but the one whose error trade-off is most appropriate for the real cost of missed fraud versus false alarms.

7. Conclusion

This project demonstrates a complete data mining workflow for rare event fraud detection. After preprocessing the Kaggle credit card dataset, we analyzed feature patterns, compared multiple classification models, selected Random Forest based on validation performance, tuned the fraud decision threshold on validation data, and evaluated the final configuration once on the held-out test split. The final model achieved strong precision with moderate recall, catching 69 of 95 fraud cases in the test set while producing only 5 false positives. The results show that fraud detection must be evaluated with precision, recall, F1, PR-AUC, and confusion matrix counts because high accuracy alone can hide missed fraud in severely imbalanced data.

Future work could improve the system by selecting thresholds with explicit fraud and review costs, adding stronger hyperparameter tuning and probability calibration, and monitoring deployed performance over time as fraud patterns change.

References

[1] Kaggle. "Credit Card Fraud Detection" dataset. Retrieved March 21, 2026, from https://www.kaggle.com/dalpozz/creditcardfraud

[2] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating Probability with Undersampling for Unbalanced Classification," IEEE Symposium on Computational Intelligence and Data Mining (CIDM), 2015. https://doi.org/10.1109/SSCI.2015.33